



Machine Learning for Petroleum Engineers Using Python

Módulo 3 · Clase 2

Clasificación: Predecir una Etiqueta, Decidir con Costo



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos

✓ Clase anterior:

- ✓ Relación entre variables y correlación
- ✓ Regresión lineal: predecir **un número**
- ✓ Métricas RMSE, MAE, R^2
- ✓ Train/test y overfitting

*Predijimos el sónico: la respuesta era un número
(84.2 μ s/ft).*

▶▶ Hoy:

- El **mapa completo** del ML: supervisado y no supervisado
- **Clasificación**: cuando la respuesta es una **etiqueta**
- Regresión **logística**
- ★ **Métricas y costo de negocio**: el corazón de la clase

*Pregunta de hoy: ¿esta roca es **reservorio** o **sello**?*

El Mapa del Machine Learning

Dónde encaja cada cosa

Aprendizaje Supervisado: Aprender con Respuestas

“Supervisado” porque los ejemplos vienen con la respuesta correcta, como un profesor que corrige.

- Es como aprender con un **cuaderno de ejercicios resueltos**: cada ejemplo trae el resultado al final.
- El computador compara su intento con la respuesta, mide el error y **se corrige**.
- ✓ Es lo que hicimos la clase pasada: le dimos miles de profundidades **con su sónico ya medido**.

Ejemplos petroleros:

- Predecir el sónico faltante
- Predecir producción de un pozo
- Decir si una roca es reservorio
- Detectar si una bomba va a fallar

En todos: alguien ya midió o etiquetó la respuesta en los datos históricos.

Aprendizaje No Supervisado: Encontrar Patrones Solo

Aquí no hay respuesta correcta. Le pedimos al computador que encuentre estructura por sí mismo.

- ≡ Le damos una caja de datos y le decimos: **“agrupa lo que se parezca”**.
- ? Nadie le dijo cuántos grupos hay ni cómo se llaman; los **descubre** él.
- 👤 Después el **ingeniero interpreta**: “este grupo parece arena limpia, este otro parece arcilla”.

Ejemplos petroleros:

- Agrupar pozos con comportamiento parecido
- Encontrar electrofacies en los registros
- Detectar mediciones anómalas

Es el **Módulo 4** del programa (cluster analysis).

Supervisado vs No Supervisado

La pregunta que los separa: ¿mis datos históricos traen la respuesta?

	Supervisado	No supervisado
¿Hay respuesta?	Sí: cada ejemplo trae su etiqueta o su número	No: solo tenemos las mediciones
¿Qué hace?	Predice un valor nuevo	Agrupar o resume lo que hay
¿Cómo sé si sirve?	Comparo mi predicción con la realidad	Lo juzga el experto: ¿los grupos tienen sentido?
Ejemplo	“predice el sónico” / “¿es reservorio?”	“agrupa estas 500 profundidades”

💡 Hoy seguimos en **supervisado**. Pero cambia el tipo de respuesta.

Dentro de Supervisado: ¿Número o Etiqueta?

Todo se decide con una pregunta: ¿qué tipo de respuesta busco?



REGRESIÓN

“¿cuánto?”

La respuesta es un **número** en una escala continua.

El sónico es 84.2 μ s/ft

El pozo produce 1250 bbl/día ✓ Clase anterior



CLASIFICACIÓN

“¿cuál?”

La respuesta es una **etiqueta** de una lista de opciones.

Esta roca es arenisca

Esta bomba va a fallar ★ Hoy

Test rápido: ¿tiene sentido decir “me equivoqué por 3”? Si sí → regresión. Si la respuesta es “acerté o no” → clasificación.

El Mapa Completo, con Ejemplos de Campo

	Regresión (un número)	Clasificación (una etiqueta)
Petrofísica	reconstruir el sónico faltante	¿arenisca o lutita?
Producción	¿cuántos barriles dará mañana?	¿el pozo seguirá activo o hay que cerrarlo?
Equipos	¿cuántas horas le quedan a la bomba?	¿va a fallar en los próximos 30 días?
Perforación	¿cuánto tardará perforar la sección?	¿habrá pega de tubería?

💡 El **mismo problema** puede plantearse de las dos formas. La elección depende de **qué decisión** va a tomar quien reciba el resultado.

El Problema de Hoy

¿Reservorio o sello?

La Pregunta que Vale Millones

Tenemos los registros de un pozo nuevo. Hay que decidir qué intervalos completar y cañonear.

- ≡ La **arenisca** puede almacenar y dejar fluir hidrocarburos: es **roca reservorio**.
- ⊘ La **lutita** (arcilla) no deja fluir nada: es **roca sello**.
- ? Un geólogo lo interpreta a mano, metro por metro. Un pozo tiene **miles** de metros.
- 🤖 ¿Puede un modelo hacer una primera pasada a partir de los registros?

Los dos errores posibles:

Cañonear una zona seca

gastamos en una completación que no produce.

Saltarse una zona productiva

dejamos petróleo en el subsuelo.

No cuestan lo mismo. Volveremos a esto — es el corazón de la clase.

El Dataset: FORCE 2020 (Mar del Norte)

Registros de 11 pozos noruegos con la litología interpretada por geólogos.

Python

```
lito = pd.read_csv(url)
lito.shape
```

>_ Output

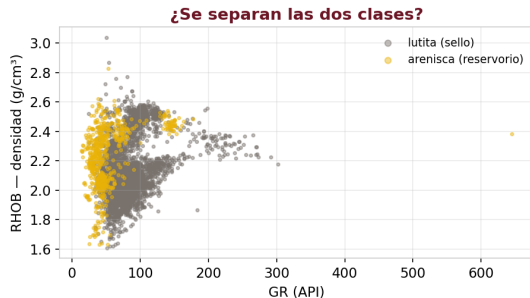
```
(62792, 9)
```

- Dataset abierto de la competencia **FORCE 2020** (Noruega), del mismo vecindario de Volve.
- Las etiquetas **no salen de una fórmula**: las puso un intérprete humano. Eso las hace confiables...y también imperfectas.

Columna	Qué mide	Por qué ayuda
GR	radioactividad natural	la arcilla es radioactiva, la arena no
RHOB	densidad de la roca	la arena porosa pesa menos
NPHI	porosidad de neutrón	la arcilla retiene agua ligada
DTC	sónico (el de la clase pasada)	la arena compacta transmite rápido
RDEP	resistividad profunda	el fluido de los poros
LITH	la etiqueta	Sandstone o Shale

Explorar: ¿Se Separan las Dos Clases?

Antes de modelar: ¿las mediciones distinguen una roca de la otra?



- ✓ Buenas noticias: los colores **ocupan zonas distintas**. La arenisca vive a GR bajo.
- ⚠ Malas noticias: **se traslapan** en el medio. No hay una línea que separe perfecto.
- ➔ Por eso el modelo no dará certezas: dará **probabilidades**.

Una Alerta Temprana: el Desbalance

¿Cuántos ejemplos hay de cada clase? La respuesta cambia todo lo que viene después.

Python

```
lito["LITH"].value_counts()
```

>_ Output

Shale	52011
Sandstone	10781

⚠ Guarden este número

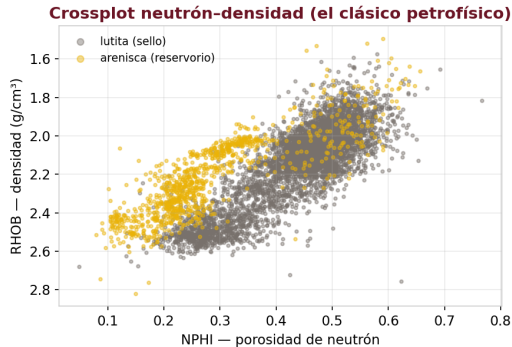
Un “modelo” que responda **siempre “lutita”**, sin mirar ningún dato, acertaría el **83 %** de las veces.

Volveremos a esto cuando hablemos de métricas.

83 % lutita, 17 % arenisca. En geología es lo normal: la mayor parte de una columna sedimentaria es arcilla.

El Crossplot Neutrón-Densidad

El gráfico que un petrofísico mira todos los días: NPHI contra RHOB.

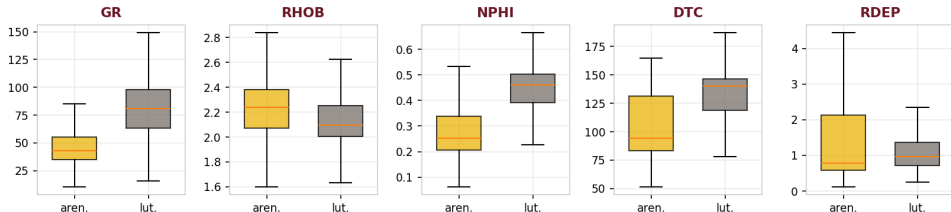


- El eje de densidad va **invertido** (convención: la roca ligera arriba).
- ✓ La **arenisca** se agrupa a la **izquierda**: poca respuesta de neutrón.
- La **lutita** se corre a la derecha: la arcilla retiene agua ligada y **finge** porosidad.
- ⚠ Otra vez: separan... pero con una **zona de mezcla** en el medio.

¿Cuánto Separa Cada Registro?

Un vistazo rápido a las cinco variables a la vez.

¿Cuánto separa cada registro las dos rocas?

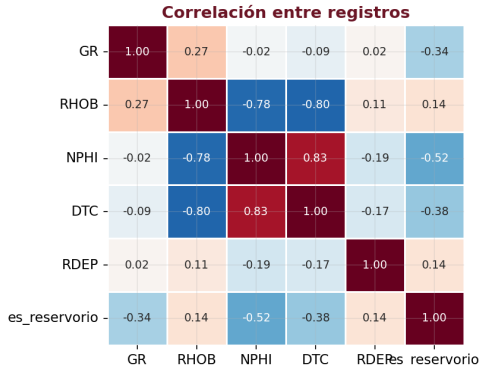


✓ GR, NPHI y DTC: las cajas están **claramente desplazadas** → separan bien.

✗ RHOB y RDEP: las cajas casi se superponen → aportan poco **por sí solas**.

La Matriz de Correlación

Además de mirar el target, conviene ver cómo se relacionan las variables entre sí.



→ Con el target: NPHI es la más ligada (-0.52), luego DTC y GR.

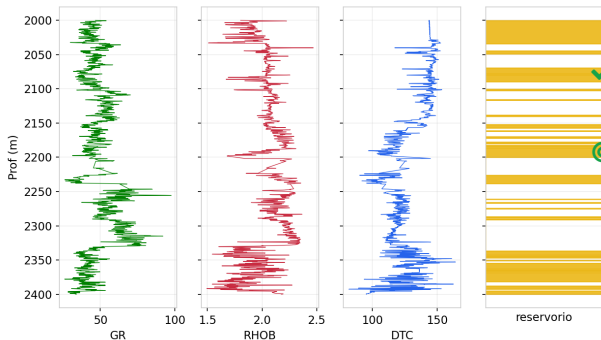
🔗 Entre features: NPHI y DTC están muy correlacionadas **entre ellas** — las dos hablan de porosidad.

⚠ Eso se llama **colinealidad**: información repetida. No arruina el modelo, pero vuelve **inestables** los coeficientes.

Cómo se Ve Esto en un Pozo Real

Los mismos datos, en el formato en que los lee un geólogo: contra la profundidad.

Pozo 15/9-13: los registros y la litología interpretada



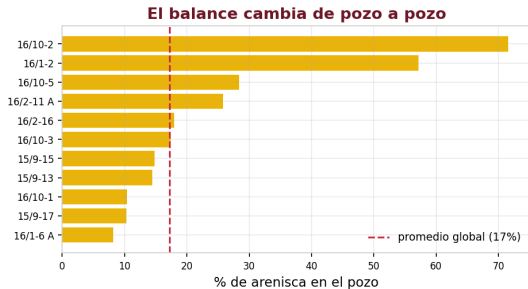
→ Las tres primeras pistas son los **registros**; la cuarta, la **interpretación del geólogo**.

✓ Miren la coincidencia: donde el GR **baja**, aparece una barra amarilla.

🎯 **Eso** es lo que el modelo tiene que aprender: reproducir esa cuarta pista a partir de las tres primeras.

Un Detalle que Casi Nadie Mira: el Balance por Pozo

El 17 % global esconde diferencias enormes entre pozos.



- Hay pozos con **8 %** de arenisca y otros con **72 %**. Son geologías distintas.
- ⚠ Al partir train/test **al azar**, mezclamos profundidades del mismo pozo en ambos lados: el modelo ya “conoce” ese pozo.
- ✓ Lo más honesto sería **dejar pozos completos por fuera** para probar. Es una decisión metodológica real.

De la Recta a la Probabilidad

Regresión logística

Preprocesamiento: el Modelo No Entiende Texto

Nuestro target es una palabra. Antes de entrenar hay que volverlo número.

como viene el dato

```
LITH
Sandstone
Shale
Shale
Sandstone
```

como lo necesitamos

```
LITH      es_reservorio
Sandstone 1
Shale     0
Shale     0
Sandstone 1
```

✗ **LogisticRegression** no puede multiplicar la palabra "Shale" por un coeficiente.

✓ **Binarizar**: convertir dos categorías en 0 y 1.

💡 Es parte del **preprocesamiento**: todo lo que hay que hacerle al dato **antes** de que el modelo lo pueda usar (junto con estandarizar y limpiar).

Binarizar: Cómo se Hace y Por Qué ImportaCuál es el 1

Python

```
lito["es_reservorio"] = (  
    lito["LITH"] == "Sandstone"  
).astype(int)
```

La comparación da True/False; .astype(int) lo vuelve 1/0. **¿Y si**

hubiera más de dos clases?

Con 3 o más categorías se usa **one-hot encoding** (pd.get_dummies):
una columna 0/1 por categoría.

💡 Regla: la **clase positiva** (1) es siempre **el evento raro que queremos cazar** — la zona productiva, la falla del equipo, la fuga.

⚠ La decisión que define todo

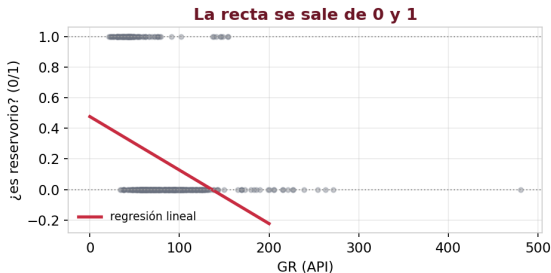
Elegimos **arenisca = 1** porque es lo que **queremos detectar**: la **clase positiva**.

Eso fija el significado de todo lo que viene: **recall** será
“% de **areniscas** encontradas”.

Si pusiéramos lutita = 1, las mismas métricas medirían
otra cosa.

¿Por Qué No Usar la Recta de la Clase Pasada?

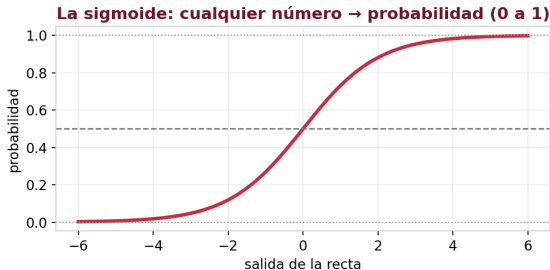
Podríamos poner 0 = lutita y 1 = arenisca, y ajustar una recta. Veamos qué pasa.



- ✗ La recta **no se detiene** en 0 ni en 1: predice 1.3 o -0.2.
- ❓ ¿Qué significa “1.3 de arenisca”? Nada.
- 💡 Necesitamos algo que **siempre** caiga entre 0 y 1, para leerlo como **probabilidad**.

La Solución: la Curva Sigmoide

Una función que toma cualquier número y lo aplasta entre 0 y 1.



- Números muy negativos $\rightarrow$ probabilidad cercana a **0**.
- Números muy positivos $\rightarrow$ cercana a **1**.
- En el medio, una transición **suave**: ahí viven los casos dudosos.

Regresión logística = la misma recta de siempre, pasada por esta curva.

Entrenar el Modelo: el Mismo Patrón de sklearn

Cambia el modelo, no el flujo. Es el mismo `fit` de la clase pasada.

Python

```
from sklearn.linear_model import
    LogisticRegression

X = lito[["GR", "RHOB", "NPHI", "DTC", "RDEP"]]
y = lito["es_reservorio"]    # 1 = arenisca

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42)

modelo = LogisticRegression(max_iter=1000)
modelo.fit(X_tr, y_tr)
```

- El target debe ser **0/1**, no texto.
- Igual que antes: **estandarizar** las features ayuda (escalas muy distintas).
- ✓ `.fit()`, `.predict()`, `.score()`: **idéntico** a la regresión lineal.

¿Qué es la Regresión Logística?

Pese al nombre, no sirve para predecir números: es el modelo base para clasificar.



- Por dentro es **la misma recta** de la clase pasada: cada registro aporta con su peso ($m_1GR + m_2RHOB + \dots$).
- La diferencia: el resultado **pasa por la sigmoide**, que lo convierte en una **probabilidad**.
- ✓ Se llama “regresión” porque **internamente** hace una regresión; lo que **entrega** es una clasificación.

¿Para Qué se Usa? ¿Por Qué Empezar por Ella?

Es el primer modelo que se prueba en cualquier problema de clasificación — y muchas veces el que queda.

✓ Sus fortalezas:

- 👁 **Se puede explicar:** cada coeficiente dice cómo influye cada registro. Auditable ante un cliente o un regulador.
- ⚡ **Rápida** y estable, incluso con pocos datos.
- 📊 Sirve de **línea base:** si un modelo complejo no la supera, no vale la pena su complejidad.

✗ Sus límites:

- ⚠ Separa las clases con una **frontera recta**. Si la frontera real es curva, se queda corta.
- ⚠ Le afectan los **outliers** y las escalas muy distintas (por eso estandarizamos).

Para fronteras curvas: árboles y bosques — las próximas clases.

Usos típicos en la industria: litología · ¿pozo productor o seco? · ¿fallará el equipo? · ¿hay riesgo de pega de tubería?

Leer el Modelo: ¿Qué Aprendió?

Su gran ventaja: podemos abrirlo y ver en qué se fijó (con las features estandarizadas).

Python

```
pd.Series(modelo.coef_[0],  
          index=X.columns).round(2)
```

>_ Output

```
NPHI      -3.36  
RHOB      -2.59  
GR         -1.65  
DTC        -0.84  
RDEP        0.01
```

- Coeficiente **negativo** = ese registro alto **empuja hacia lutita**.
- ✓ GR negativo: mucha radioactividad → arcilla. **Coincide con la geología.**
- ✓ NPHI y RHOB son los que más pesan.
- ✗ RDEP ≈ 0 : no aporta a distinguir la litología.

💡 Que el modelo “piense” como la física **es una señal de que no está memorizando ruido**.

Lo Que Realmente Devuelve: Probabilidades

El modelo no dice “arenisca”. Dice “78 % de probabilidad de arenisca”.

Python

```
proba = modelo.predict_proba(X_te)[: , 1]  
proba[:4].round(2)
```

>_ Output

```
[0.03  0.78  0.41  0.95]
```

Python

```
pred = (proba >= 0.5).astype(int)
```

→ Para dar una respuesta hay que fijar un **umbral**: por defecto **0.5**.

⚠ Ese 0.5 es una **convención**, no una ley.

★ **Moverlo es la decisión más importante de la clase.**
Lo haremos al final.

Las Métricas

Aquí se decide si el modelo sirve

Antes de las Fórmulas: el Detector de Gas

Ustedes ya saben evaluar un clasificador: conviven con uno todos los días.

Detector que NUNCA suena

Como casi nunca hay fuga, acierta el **99.9 %** de los días.

Accuracy altísima... y es un adorno: el día de la fuga no avisa.

Detector que suena SIEMPRE

No se le escapa ninguna fuga jamás.

Pero nadie le cree: tantas falsas alarmas que la cuadrilla lo ignora.

- Nadie evalúa un detector de gas por su **% de aciertos**. Se evalúa por **cuántas fugas detecta y cuántas veces molesta en falso**.
- ✓ Esas dos preguntas tienen nombre técnico: **recall** y **precision**. Ya las entienden — ahora les ponemos el nombre.

Accuracy: la Métrica que Engaña

Accuracy = % de aciertos. Es la primera que todos miran...y la más peligrosa.

Python

```
modelo.score(X_te, y_te)
```

>_ Output

```
0.944
```

⚠ Recuerden el desbalance

Responder **siempre** “lutita”, sin mirar nada, da **82.8 %**.

Nuestro modelo “excelente” solo mejora **12 puntos** sobre no pensar.

94.4 % de aciertos. **Excelente**, ¿no?

💡 Con clases desbalanceadas, la accuracy **esconde** lo que de verdad importa: ¿encontramos las zonas de reservorio, que son las escasas?

Abrir la Caja: la Matriz de Confusión

En vez de un número, los cuatro resultados posibles — con nombre de negocio.

Matriz de confusión (test)

	predije sello	predije reservorio
ES sello	Correcto (sello) 12680	FALSO POSITIVO cañoneo en seco 323
ES reservorio	FALSO NEGATIVO zona perdida 563	Correcto (reservorio) 2132

- ✓ **Aciertos** (diagonal): 12 680 sellos y 2 132 reservorios bien identificados.
- ✗ **323 falsos positivos**: dijimos reservorio y era sello → *cañoneo en seco*.
- ✗ **563 falsos negativos**: dijimos sello y era reservorio → *zona productiva perdida*.

La accuracy sumaba todo esto en un solo número y **escondía** los 563.

Las Dos Preguntas: Precision y Recall

Dos métricas que responden preguntas distintas sobre la misma matriz.

PRECISION = 0.87

*“De todas las veces que dije **reservorio**, ¿cuántas acerté?”*

Mide si podemos **confiar** cuando el modelo dice que sí.

Importa cuando actuar en falso es caro (cañonear en seco).

RECALL = 0.79

*“De todas las zonas que **eran** reservorio, ¿cuántas encontré?”*

Mide cuánto se nos **escapa**.

Importa cuando perderse algo es caro (dejar petróleo en el subsuelo).

Nuestro recall de **0.79** significa: de cada 10 zonas de reservorio, **se nos escapan 2**.

De Dónde Sale Cada Métrica

Ambas salen de la misma matriz. La diferencia es qué se toma como referencia.

RECALL = de las que ERAN

ES sello	563	2 132
ES reservorio	563	2 132

mira la FILA: $2132 / (2132 + 563) = 0.79$

PRECISION = de las que DIJE

12 680	323
563	2 132

predije sello

predije reservorio

mira la COLUMNA: $2132 / (2132 + 323) = 0.87$

💡 Truco para no confundirlas: **recall** mira la **realidad** (¿cuánto de lo que existía encontré?);
precision mira **mis anuncios** (¿cuánto de lo que dije era cierto?).

¿Cuál Priorizar? Depende del Daño

No hay una mejor: hay una que importa más en su problema.

Situación	Priorizar	Por qué
Detectar zonas de reservorio	Recall	perderse una zona productiva cuesta reservas para siempre
Alarma de fuga de gas	Recall	no detectar una fuga es un evento de seguridad
Recomendar un workover de \$2M	Precision	si nos equivocamos gastamos el presupuesto en vano
Parar la producción por alerta	Precision	cada parada en falso cuesta producción diferida

⚠ Subir una **casi siempre** baja la otra. No se puede maximizar ambas.

✓ La pregunta correcta no es “¿cuál es mejor?” sino “¿cuál error hace más daño aquí?”

F1 y las Métricas en Código

F1 combina precision y recall en un solo número (útil para comparar modelos).

Python

```
from sklearn.metrics import (accuracy_score,
                             precision_score, recall_score, f1_score,
                             confusion_matrix)

pred = modelo.predict(X_te)

print(accuracy_score(y_te, pred))
print(precision_score(y_te, pred))
print(recall_score(y_te, pred))
print(f1_score(y_te, pred))
```

>_ Output

```
0.944    accuracy
0.868    precision
0.791    recall
0.828    f1
```

F1 castiga los desequilibrios: si el recall es bajo, el F1 baja aunque la precision sea alta.

¿Cuál Métrica Uso? Criterios y Valores

Métrica	Cuándo elegirla	Qué es “acceptable”
Accuracy	solo si las clases están balanceadas	debe superar claramente el % de la clase mayoritaria
Precision	cuando una falsa alarma cueste cara	> 0.8 en decisiones operativas
Recall	cuando perderse un caso cueste caro	> 0.9 si el caso perdido es crítico (seguridad, reservas)
F1	comparar modelos con clases desbalanceadas	> 0.7 razonable · > 0.85 muy bueno

⚠ **Nunca** reportar una sola métrica. Precision y recall **siempre** van juntas.

→ Y todas, **sobre el test**.

El Costo de Negocio

Lo que separa a un ingeniero de un operador de librerías

Los Dos Errores NO Cuestan lo Mismo

Hasta ahora contamos errores. Ahora vamos a ponerles precio.

FALSO POSITIVO

“dije reservorio, era sello”

Cañoneamos y completamos un intervalo que no produce.

Costo: el trabajo de completación desperdiciado. Es **dinero gastado**, medible, y se detecta rápido.

FALSO NEGATIVO

“dije sello, era reservorio”

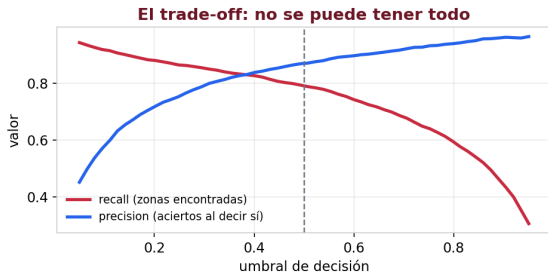
Nunca completamos esa zona.

Costo: las reservas que esa zona habría producido **durante toda la vida del pozo**. Y **nadie se entera**: no aparece en ningún reporte.

En este problema, **perder una zona productiva** suele costar **mucho más** que un cañoneo en seco.

El Umbral: la Perilla que Casi Nadie Toca

Bajar el umbral = ser más generoso al declarar reservorio. Encontramos más...y nos equivocamos más.



- ↓ Umbral **bajo**: recall sube (encuentro casi todo), precision baja (muchas falsas alarmas).
- ↑ Umbral **alto**: precision sube (cuando digo sí, acierto), pero se me escapan zonas.
- ★ **No existe el umbral “correcto” en abstracto.** Depende de qué error cuesta más.

Ponerle Precio: la Matriz de Costos

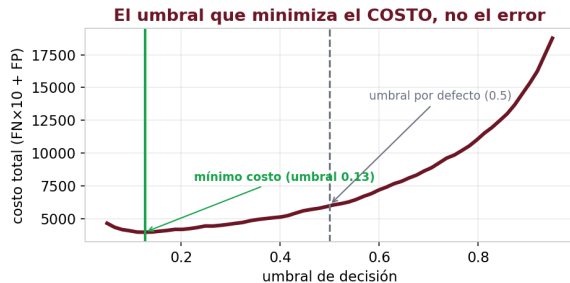
Supongamos, con el equipo de yacimientos, que perder una zona cuesta 10 veces un cañoneo en seco.

Umbral	Zonas perdidas (FN)	Cañoneos en seco (FP)	Costo total ($FN \times 10 + FP$)
0.50 (por defecto)	563	323	5 953
0.40	469	434	5 124
0.30	400	603	4 603
0.25	368	760	4 440
0.20	324	939	4 179

💡 Al bajar el umbral aceptamos **más** cañoneos en seco (323 → 939) para rescatar **239 zonas productivas**. En dinero, el cambio **conviene**.

El Umbral que Minimiza el Costo

Si graficamos el costo total contra el umbral, aparece el punto óptimo.



- El umbral por defecto (**0.5**) cuesta **5 953**.
- ✓ El umbral óptimo (**0.13**) cuesta **3 962**.
- 🏆 **33 % menos costo** — sin cambiar el modelo, sin más datos: solo moviendo una perilla.

El Veredicto: Peor Modelo, Mejor Negocio

	Umbral 0.5 (por defecto)	Umbral 0.13 (por costo)
Accuracy	0.944	0.895
Recall	0.791	0.905
Zonas perdidas	563	257
Costo total	5 953	3 962

El segundo modelo tiene **peor accuracy**. Un reporte automático diría que es **inferior**.
Y sin embargo es el que **le conviene a la empresa**.

★ Eso es lo que separa al ingeniero que entiende el negocio del que solo corre `.fit()` y reporta la accuracy.

Cómo se Hace Esto en la Práctica

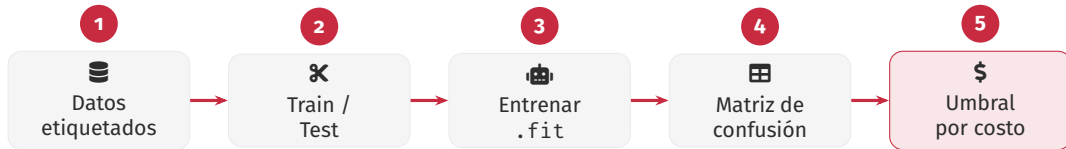
El paso que ningún curso de ML enseña y todo proyecto real necesita.

1. **Preguntar, no asumir.** Sentarse con yacimientos, producción o finanzas: *¿cuánto cuesta cada error?*
Casi nunca está escrito.
2. **Basta la proporción.** No hace falta el número exacto: con saber que un error cuesta ~ 10 veces el otro, ya se puede decidir.
3. **Elegir el umbral con esa proporción**, no con el 0.5 de fábrica.
4. **Reportar en el lenguaje del que decide:** no “recall 0.91” sino “*recuperamos 306 zonas que antes se perdían, a cambio de 1069 cañoneos adicionales*”.

Cierre

Lo que se llevan hoy

El Flujo de un Problema de Clasificación



- ✓ Los pasos 1–3 son **idénticos** a la regresión de la clase pasada.
- ★ Los pasos **4 y 5** son los que hacen la diferencia — y los que casi nadie hace.

Lo Que Aprendimos Hoy

Conceptos

- ✓ Supervisado vs no supervisado
- ✓ Regresión (¿cuánto?) vs clasificación (¿cuál?)
- ✓ Por qué la recta no sirve para sí/no
- ✓ Sigmoide, probabilidad y **umbral**
- ✓ Desbalance: por qué la accuracy engaña
- ✓ Matriz de confusión, precision, recall, F1
- ★ **Matriz de costos y umbral óptimo**

Herramientas

- ✗ LogisticRegression
- ✗ .predict_proba()
- ✗ confusion_matrix
- ✗ precision_score, recall_score, f1_score

La idea para llevarse:

Un modelo no se evalúa con una métrica. Se evalúa con **la decisión que habilita y lo que cuesta equivocarse.**

Lo Que Sigue

Ya tienen los dos tipos de problema supervisado. El módulo continúa.

- ➔ **Modelos más flexibles** — cuando la recta y la sigmoide se quedan cortas: árboles y bosques aleatorios.
- ➔ **Módulo 4: aprendizaje no supervisado** — agrupar sin etiquetas (*cluster analysis*), con David Ponce.
- ➔ **Módulo 5** — ML aplicado a yacimientos y producción.

Y en todos: la misma pregunta final — ¿qué decisión habilita este modelo, y cuánto cuesta equivocarse?

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

SLB Ecuador · UDLA · 2026